E-Signature Elections Canada

Solution Architecture & Design Document

Core Platform, Broker API, & M365 Office Files Templating (integration)

Version	2.0
Last Updated	March 2026
Document Owner	Platform Engineering Team
Author	Frederick Pearson
Status	Draft for Review

Elections Canada — Platform Engineering Team

Document Control

Version	Date	Author	Changes
1.0	February 2026	Frederick Pearson	Initial release – Core broker service architecture
2.0	March 2026	Frederick Pearson	Added Document Templating & Power Pages Portal feature (Part 3); updated data model, integration flows, and security model accordingly

Table of Contents

Part 1 – Executive Summary & Architecture Overview

1. Executive Summary
2. Architecture Overview
3. Access Control & Identity Management

Part 2 – Data & Integration Architecture 4. Data Architecture 5. Integration Architecture

Part 3 – Document Templating & Power Pages Portal 6. Feature Overview 7. Portal Architecture & Design 8. Document Processing Pipeline 9. Templating Security & Access Control 10. Templating Data Model Extensions 11. Templating Implementation & Deployment

Part 4 – Security & Monitoring 12. Security Architecture 13. Monitoring, Logging & Audit

Part 5 – Operations & Compliance 14. Data Retention & Disposition 15. Client Onboarding Process 16. Environment Strategy 17. Network Architecture 18. Disaster Recovery & Business Continuity 19. Compliance & Governance

Part 6 – Annexes

- Annex A: Environment Configuration
- Annex B: Security Controls Mapping (ITSG-33)
- Annex C: API Specifications
- Annex D: Power Pages Portal Configuration
- Annex E: Document Conversion Specifications
- Annex F: Field Mapping Schema
- Annex G: Troubleshooting Guide

Part 1: Executive Summary & Architecture Overview

1. Executive Summary

1.1 Purpose

This document describes the solution architecture for the **ESign Elections Canada Broker Service**, a digital signature platform that provides Elections Canada (EC) with secure, compliant, and auditable electronic signature capabilities via Nintex AssureSign.

Version 2.0 incorporates the **Document Templating & Power Pages Portal** feature, which extends the platform to allow EC staff to upload Word and PDF documents directly through a self-service web portal, map signature fields, and submit documents for e-signature — without requiring IT involvement or pre-built Nintex templates.

1.2 Scope

The architecture encompasses the following functional domains:

Broker Service Layer – Microsoft Power Platform (Dataverse) acting as API gateway and middleware between client applications and Nintex AssureSign.

Client Integration Layer – Custom connectors and OData endpoints for programmatic client consumption by internal applications and services.

Document Templating Layer *(new in v2.0)* – Power Pages portal allowing authenticated EC staff to upload documents, map signature fields, and initiate e-signature workflows self-service.

External Integration – Nintex AssureSign API integration for digital signature delivery and lifecycle management.

Security & Compliance – Protected B controls, ITSG-33 alignment, audit logging, PII management, and Privacy Impact Assessment coverage.

Operations – Monitoring, logging, backup, disaster recovery, and client onboarding procedures.

1.3 Key Architectural Principles

Principle	Description	Implementation
Support for Multiple Business Units	Logical isolation between client applications and services	Row-level security (RLS) in Dataverse; business units and security roles for API access governance
API-First	OData/REST endpoints as primary integration method	Dataverse Web API and Custom Connectors
Zero Trust	Never trust, always verify	OAuth 2.0, service principals, conditional access
Defense in Depth	Layered security controls	Network, identity, application, and data encryption layers
Audit Everything	Complete audit trail	Dataverse audit logs and Azure Monitor
Least Privilege	Minimal necessary permissions	Custom security roles and Entra ID RBAC
Self-Service	Reduced IT dependency for routine signature workflows	Power Pages portal with guided document upload and field mapping

1.4 High-Level Architecture

The platform architecture consists of four primary interaction layers:

Client Layer – Internal applications and services interact with the broker through the Custom Connector or directly via the Dataverse Web API (OData). Beginning with v2.0, authenticated EC staff also interact through the Power Pages portal.

Broker Service Layer – Power Platform (Dataverse) serves as the central orchestration hub, managing envelope lifecycle, Row-Level Security, Power Automate workflow execution, and all integration with Nintex.

Supporting Azure Services – Azure Key Vault (secrets management), Azure Monitor (logging and alerting), and Entra ID (authentication and authorization).

External SaaS – Nintex AssureSign is the underlying digital signature platform, accessed exclusively via the broker's Power Automate orchestration flows.

1.5 Technology Stack

Layer	Technology	Version	Purpose
Platform	Microsoft Power Platform	Latest	Broker service host
Database	Dataverse	9.2+	Data store and API
Orchestration	Power Automate	N/A	Workflow automation
Identity	Microsoft Entra ID	Latest	Authentication and authorization
Secrets	Azure Key Vault	Latest	Credential management
Monitoring	Azure Monitor	Latest	Logging and alerting
API Gateway	Dataverse Web API	9.2+	OData endpoints
External SaaS	Nintex AssureSign	3.7+	Digital signature platform
Portal *(v2.0)*	**Power Pages**	**Latest**	**Self-service document upload portal**
Conversion *(v2.0)*	**LibreOffice Online (Collabora)**	**Latest**	**Word-to-PDF conversion service**
PDF Processing *(v2.0)*	**PDF.js and Custom Field Parser**	**Latest**	**Signature field extraction from PDF documents**

2. Architecture Overview

2.1 Architectural Patterns

2.1.1 Broker Pattern

The solution implements a broker architectural pattern in which:

1. Client services and the Power Pages portal interact with the broker via the standard OData API (Dataverse Web API).
2. The broker manages complexity, approval workflows, integration layer governance, and file handling (Office and PDF documents as API middleware).
3. Nintex AssureSign provides the underlying digital signature capabilities.

The key purposes served by this pattern are centralized Nintex license management, consistent security controls across all client types, simplified client integration, and a unified audit and compliance trail.

2.1.2 Client Service to Broker Service Architecture

Each client application or service is assigned a unique Service Principal in Entra ID. This principal is mapped to a Dataverse Application User, which is assigned to a custom security role. Dataverse enforces Row-Level Security via the `ownerid` field, ensuring that each client can only read and write their own records.

The v2.0 portal introduces a parallel authentication model: authenticated EC staff users (human identities) interact with the broker through Power Pages, with their Dataverse Contact record serving as the owner of envelopes created through that channel.

Isolation Mechanisms Summary:

Mechanism	Implementation	Security Level
Identity (programmatic clients)	Unique service principal per client	Entra ID tenant-level
Identity (portal users)	Entra ID SSO mapped to Dataverse Contact	Entra ID tenant-level
Data	Owner-based RLS in Dataverse	Row-level
Network	Shared PaaS with optional IP restrictions	Tenant-level
Logging	Separate Log Analytics workspaces per client	Subscription-level

2.2 Dataverse as Integration Platform

Dataverse is the core integration hub for the broker service for the following reasons:

OData Protocol Support – Dataverse exposes an industry-standard REST API supporting `$filter`, `$select`, `$expand`, standardized OAuth 2.0 authentication, native pagination, and batch operations.

Web API Capabilities – Full RESTful CRUD operations, custom actions (e.g., `cs_SendEnvelope`), webhooks for real-time integration, and SDK support across C#, JavaScript, and Python.

Built-in Security – Row-level security, column-level security, native audit logging, and AES-256 data encryption at rest and in transit via TLS 1.2+.

Power Platform Integration – Native Power Automate triggers, Power Apps data source support, custom connector compatibility, and Power Pages portal data access via the Web API.

3. Access Control & Identity Management

3.1 Authentication Architecture

The broker employs a multi-step token-based authentication model:

1. The client application or portal requests a token from the Entra ID token endpoint.
2. Entra ID authenticates the requestor (service principal credentials for programmatic clients; user credentials plus MFA for portal users) and generates an OAuth 2.0 access token.
3. The client presents the token with every API call to Dataverse.
4. Dataverse validates the token, maps the caller to the corresponding Application User or Contact record, checks security role permissions, and applies Row-Level Security filters automatically.

3.2 Authentication Methods

3.2.1 Service Principal (Recommended for Programmatic Clients)

This method is used for Power Automate flows, custom applications, and scheduled jobs. Each onboarded client receives a dedicated App Registration in Entra ID, a corresponding Service Principal, a client secret stored in Azure Key Vault, and Dataverse API permissions granted with admin consent. Tokens are acquired using the OAuth 2.0 client credentials grant and must target the broker Dataverse environment as the resource scope.

3.2.2 Managed Identity (For Azure-Hosted Applications)

Azure Functions, Logic Apps, and Virtual Machines can authenticate using system-assigned managed identities, eliminating the need to manage client secrets in code. The managed identity principal is granted the Dynamics 365 User role and acquires tokens via the Azure

Instance Metadata Service endpoint.

3.2.3 Power Pages Portal SSO (For Human Users — v2.0)

EC staff accessing the Document Templating portal authenticate via OpenID Connect against the Elections Canada Entra ID tenant. If a valid browser session already exists (the user is already signed in to their EC account), authentication is silent with no re-prompt. Upon first authentication, a Dataverse Contact record is automatically created or looked up and linked to the portal session. All envelopes created via the portal are owned by the authenticated Contact, and RLS is enforced identically to programmatic clients.

3.3 Authorization Model

Entra ID permissions are mapped to Dataverse Application Users, who are assigned security roles. Security roles grant specific privileges on each Dataverse table. Owner-based Row-Level Security then filters data access so each caller sees only records they own.

3.4 Security Roles Configuration

Client Access Role – Grants the minimum privileges required for a client to create and manage their own signature envelopes.

Privilege	cs_envelope	cs_signer	cs_document	cs_field	cs_template	cs_apirequest
Create	Organization	Organization	Organization	Organization	None	Organization
Read	Organization	Organization	Organization	Organization	Organization	Organization
Write	Organization	Organization	Organization	Organization	None	None
Delete	Organization	Organization	Organization	Organization	None	None

Organization-level privileges are required due to Dataverse API behavior; RLS via `ownerid` restricts actual data visibility to the caller's own records.

Broker Administrator Role – Grants full access across all tables for platform administration and support purposes.

Portal Template User Role *(v2.0)* – Assigned to Power Pages Contact records. Grants the ability to create and manage envelopes, documents, signers, and fields owned by the Contact, and read-only access to public templates from the template library.

3.5 Conditional Access Policies

Policy Name	Conditions	Controls
Require MFA for Admins	User role = Admin	Require MFA
Block Legacy Auth	Client apps = Legacy	Block access
Require Compliant Device	Device state = Not compliant	Block access
IP Restriction (Optional)	IP address not in GC range	Block access
Session Controls	All users	Sign-in frequency = 8 hours

Environment	Policy Enabled	Allowed IP Ranges	MFA Required
Production	☑	[FILL: GC IP ranges]	☑
QA	☑	[FILL: GC IP ranges]	☑
Development	☐	Any	☐

Part 2: Data & Integration Architecture

4. Data Architecture

4.1 Entity Relationship Diagram

The data model is built around the `cs_envelope` entity as the primary record, with related entities for signers, documents, signature fields, templates, API audit logs, and email notifications.

Core Relationships:

- `cs_envelope` contains one or more `cs_signer` records, one or more `cs_document` records, and zero or more `cs_field` records.
- `cs_field` records are associated with both a `cs_envelope` and a `cs_signer`, and optionally with a `cs_template` when field positions originate from a saved template.
- `cs_template` stores reusable document templates with pre-mapped field positions and can serve as the source for multiple `cs_field` records.
- `cs_apirequest` logs every Nintex API interaction for audit purposes.
- `cs_emailnotification` tracks all signing invitation and reminder emails sent.

4.2 Data Model Details

4.2.1 cs_envelope

Purpose: Primary entity representing a single e-signature request, from draft through completion.

Field	Type	Purpose	Security
ownerid	Lookup	Per-client data isolation	RLS filter
cs_status	Choice	Workflow state	Indexed
cs_nintexenvelopeid	String	External Nintex reference	Unique
cs_signinginsequence	Boolean	Sequential vs. parallel signing	v2.0 feature
cs_hidesignerinfo	Boolean	Privacy mode for signers	v2.0 feature
cs_requestbody	Memo	Full API request payload (audit)	Protected B — Admin read only
cs_responsebody	Memo	Full API response (audit)	Protected B — Admin read only

Status Lifecycle:

Status	Description	Transitions To
Draft	Being assembled by client or portal user	Pending Approval, Submitted
Pending Approval	Awaiting internal approval	Approved, Rejected
Approved	Approved and queued for Nintex	Submitted
Submitted	Sent to Nintex API	InProcess, Failed
InProcess	Signers notified by Nintex	Completed, Declined, Cancelled
Completed	All signatures obtained	None (terminal)

Status	Description	Transitions To
Declined	Signer declined to sign	None (terminal)
Cancelled	Cancelled by requestor	None (terminal)
Rejected	Internal approval rejected	None (terminal)
Failed	Technical error during submission	Submitted (retry)

4.2.2 cs_signer

Purpose: Individual signers associated with an envelope.

Field	Type	PII Classification
cs_email	String	PII
cs_fullname	String	PII
cs_phonenumber	String	PII
cs_signinglink	String	Sensitive — unique signing URL
cs_ipaddress	String	PII — audit trail
cs_language	String	Preferred language for Nintex signing experience

4.2.3 cs_document

Purpose: Files attached to an envelope for signing.

Documents are stored as Base64-encoded content in a Memo field. Base64 encoding increases file size by approximately 33% (a 10 MB PDF becomes approximately 13.3 MB). This is for transport compatibility, not encryption. The maximum supported file size is 10 MB per document.

4.2.4 cs_field

Purpose: Signature field position mappings within a document, used to precisely place signature, initial, date, text, and checkbox fields for each signer.

New in v2.0, the `cs_templateid` lookup allows field records to be associated with a saved template as well as (or instead of) a live envelope. This enables template reuse without duplicating field position data.

Additional v2.0 fields include `cs_fieldlabel` (display label), `cs_isrequired` (mandatory flag), `cs_dateformat` (for date fields), and `cs_maxlength` (for text fields).

4.2.5 cs_template *(New in v2.0)*

Purpose: Stores reusable document templates with pre-mapped field positions. Templates can be public (readable by all portal users) or private (visible only to the creator).

Field	Type	Description
cs_templateid	Guid	Primary key
cs_name	String (100)	Template display name
cs_description	Memo	Template description
cs_filename	String (255)	Original filename

Field	Type	Description
cs_filecontent	Memo	Base64-encoded PDF content
cs_filesize	Integer	File size in bytes
cs_mimetype	String (50)	Always “application/pdf”
cs_ispublic	Boolean	Available to all portal users
cs_category	Choice	Template category (Contracts, HR, Financial, Legal, Approvals, Other)
cs_fieldcount	Integer	Number of mapped fields
ownerid	Lookup	Template creator

4.2.6 cs_apirequest

Purpose: Complete audit log of all Nintex API interactions. All fields are retained for 7 years per Protected B requirements.

Field	Type	Retention
cs_requestbody	Memo	7 years
cs_responsebody	Memo	7 years
cs_statuscode	Integer	7 years
cs_timestamp	DateTime	7 years

4.3 Row-Level Security Implementation

Each client has a unique Service Principal (or, for portal users, a unique Contact record) that is mapped to a Dataverse user identity. Security roles grant Organization-level privileges, but the `ownerid` field filter is automatically applied by the Dataverse Web API to every query, ensuring each caller retrieves only their own records.

Validation Test Cases:

Test Scenario	Expected Result
Client A queries all envelopes	Returns only Client A’s envelopes
Client A queries Client B’s envelope by ID	404 Not Found
Client A attempts to update Client B’s envelope	403 Forbidden
Admin queries all envelopes	Returns all envelopes (no filter applied)

4.4 Column-Level Security

Certain fields contain sensitive data and are restricted to Broker Administrator role only:

Field	Table	Access Level	Reason
cs_requestbody	cs_envelope	Admin read only	Contains PII in API payload
cs_responsebody	cs_envelope	Admin read only	Contains Nintex tokens
cs_signinglink	cs_signer	Owner and Admin read	Unique signing access URL
cs_ipaddress	cs_signer	Admin read only	PII — tracking data

4.5 Data Encryption

Layer	Encryption Method	Key Management
At Rest	AES-256 (Dataverse TDE)	Microsoft-managed
In Transit	TLS 1.2+	Certificate pinning
Application	Base64 encoding (not encryption)	N/A
Backup	AES-256	Microsoft-managed

> **Note:** Base64 encoding of documents is used for API transport compatibility only. Documents remain readable to authorized users in Dataverse.

5. Integration Architecture

5.1 Integration Layers

The platform follows a layered integration architecture:

Client Layer – Power Automate flows, Power Apps, custom applications (.NET/Python), scheduled jobs, Azure Functions, and the Power Pages portal.

API Gateway Layer – Dataverse Web API (OData) and the Custom Connector (Power Platform) provide standardized, authenticated endpoints for all clients.

Authentication Layer – Entra ID with OAuth 2.0 and Conditional Access policies enforce identity and access controls across all entry points.

Broker Service Layer – Power Automate orchestration flows manage the envelope lifecycle, document processing, and all Nintex API interactions.

Secrets Management – Azure Key Vault stores all Nintex API credentials and portal authentication secrets.

External SaaS – Nintex AssureSign (`api.assuresign.net`) provides the digital signature platform.

5.2 Custom Connector Architecture

The Custom Connector provides a Power Platform-native interface to the broker's Dataverse OData endpoints, enabling Power Apps and Power Automate clients to interact with the broker without writing raw HTTP calls.

Authentication: OAuth 2.0 Authorization Code flow (for user-delegated access) or Client Credentials flow (for service principal access).

Operations Exposed:

Operation	HTTP Method	Dataverse Endpoint	Description
CreateEnvelope	POST	/cs_envelopes	Create draft envelope
SendEnvelope	POST	/cs_envelopes({id})/cs_SendEnvelope	Send envelope to Nintex
AddSigner	POST	/cs_signers	Add signer to envelope
UpdateSigner	PATCH	/cs_signers({id})	Modify signer

Operation	HTTP Method	Dataverse Endpoint	Description
RemoveSigner	DELETE	/cs_signers({id})	Delete signer
AddDocument	POST	/cs_documents	Attach document
RemoveDocument	DELETE	/cs_documents({id})	Remove document
GetEnvelope	GET	/cs_envelopes({id})	Retrieve envelope details
ListEnvelopes	GET	/cs_envelopes	Query envelopes
GetSigners	GET	/cs_envelopes({id})/cs_envelope_signer	Get envelope signers
ListTemplates	GET	/cs_templates	Available templates
UpdateEnvelope	PATCH	/cs_envelopes({id})	Modify or cancel envelope
DeleteEnvelope	DELETE	/cs_envelopes({id})	Delete envelope

5.3 OData Endpoint Integration

All Dataverse entities are accessible via the standard OData API at `https://{environment}.{region}.dynamics.com/api/data/v{version}/` . The API supports the full range of OData query options:

Option	Purpose	Example
$filter	Filter results	cs_status eq ‘Completed’
$select	Specify returned columns	cs_name,cs_status
$expand	Include related entities	cs_envelope_signer
$orderby	Sort results	createdon desc
$top	Limit result count	50
$skip	Pagination offset	100
$count	Include total count	true

Batch operations are also supported, allowing multiple create or update operations to be submitted as a single HTTP multipart request, reducing API call overhead for high-volume workflows.

5.4 Nintex AssureSign API Integration

API Version: 3.7 **Base URL:** `https://api.assuresign.net/v3.7`

Authentication to Nintex requires posting API credentials (username, key, and context email) to the `/authentication/apiUser` endpoint to obtain a bearer token valid for 60 minutes. The broker caches this token for 55 minutes and automatically refreshes it before expiry. All Nintex credentials are stored in Azure Key Vault and retrieved at runtime via Managed Service Identity.

Key Nintex Endpoints:

Endpoint	Method	Purpose	Broker Usage
/authentication/apiUser	POST	Obtain access token	Every 55 minutes
/submit	POST	Create and send envelope	On SendEnvelope action

Endpoint	Method	Purpose	Broker Usage
/get	GET	Retrieve envelope status	Status sync flow
/getSigningLinks	GET	Retrieve per-signer signing URLs	After submission
/cancel	POST	Cancel an active envelope	On user cancellation
/listTemplates	GET	List available Nintex templates	Daily sync
/getCompletedDocument	GET	Download signed PDF	On completion

Payload Mapping (Dataverse → Nintex):

The broker constructs the Nintex submission payload from the envelope, signer, document, and field records in Dataverse. In v2.0, when `cs_field` records exist for the envelope (i.e., when the document originated from the portal's field mapping workflow), a `Fields` array is included in the submission payload to specify exact signature field positions. When no `cs_field` records exist (standard broker flow using a pre-configured Nintex template), the `TemplateID` is used instead.

Nintex Payload Property	Source
Subject	cs_envelope.cs_subject
Message	cs_envelope.cs_message
DaysToExpire	cs_envelope.cs_daystoexpire
ReminderFrequency	cs_envelope.cs_reminderfrequency
ProcessingMode	cs_envelope.cs_signinginsequence (Sequential / Parallel)
DocumentVisibility	cs_envelope.cs_hidesignerinfo (Private / Shared)
AllowDecline	cs_envelope.cs_allowdecline
Signers[].Email	cs_signer.cs_email
Signers[].FullName	cs_signer.cs_fullname
Signers[].SignerOrder	cs_signer.cs_signerorder
Signers[].Language	cs_signer.cs_language
Documents[].FileName	cs_document.cs_filename
Documents[].FileContent	cs_document.cs_filecontent (Base64)
Fields[].Type*(v2.0)*	cs_field.cs_fieldtype
Fields[].XPosition*(v2.0)*	cs_field.cs_xposition
Fields[].YPosition*(v2.0)*	cs_field.cs_yposition
Fields[].Width*(v2.0)*	cs_field.cs_width
Fields[].Height*(v2.0)*	cs_field.cs_height
Fields[].PageNumber*(v2.0)*	cs_field.cs_pagenumber
Fields[].Required*(v2.0)*	cs_field.cs_isrequired

5.5 Broker Orchestration Flows

Flow 1: Send Envelope to Nintex (Enhanced in v2.0)

This flow is triggered by the `cs_SendEnvelope` custom action and manages the full submission lifecycle:

1. **Validate Status** – Confirm envelope status is "Draft" (or "Approved" if approval was required). Return an error if the envelope is in any other state.
2. **Validate Contents** – Confirm at least one signer and one document exist. Return specific error responses if either is missing.
3. **Approval Check** – If `cs_requiresapproval` is set, route to the approval sub-flow and update status to "Pending Approval". The flow completes here until the approver acts.
4. **Retrieve Nintex Token** – Fetch API credentials from Azure Key Vault via Managed Service Identity and obtain a Nintex bearer token.
5. **Determine Submission Mode** *(v2.0 enhancement)* – Check whether `cs_field` records exist for the envelope.
   - If **yes** (portal/template-based submission): retrieve all `cs_field` records and include them as a `Fields` array in the Nintex payload.
   - If **no** (standard broker submission): use the configured Nintex `TemplateID` and omit the `Fields` array.
6. **Build and Submit Nintex Payload** – Construct the full JSON payload and POST to `/submit`.
7. **Handle Nintex Response** – On success, update `cs_nintexenvelopeid` and set envelope status to "Submitted". On failure, set status to "Failed", log the error, and alert the administrator after 3 failed retries.
8. **Retrieve Signing Links** – Call Nintex `/getSigningLinks` and update each `cs_signer` record with the unique signing URL.
9. **Log API Request** – Create a `cs_apirequest` record capturing the full request and response for the audit trail.

Flow 2: Status Synchronization

This flow runs on a 30-minute recurrence schedule and synchronizes Nintex envelope status back to Dataverse:

1. Query all envelopes with status "Submitted" or "InProcess".
2. For each envelope, retrieve the current status from Nintex via the `/get` endpoint.
3. If the status has changed, update the Dataverse envelope record accordingly.
4. If the new status is "Completed", download the signed documents from Nintex via `/getCompletedDocument` and store them in Dataverse. Update signer statuses and notify the requestor.
5. Log all status changes to the `cs_apirequest` audit table.

5.6 Error Handling Strategy

Error Type	Handling	Retry Logic	Notification
Nintex API 5xx	Log error, mark envelope Failed	3 retries with exponential backoff	Admin alert after 3 failures
Nintex API 4xx	Log error, mark envelope Failed	No retry	Requestor notification
Token Expired	Refresh token automatically	Automatic	None
Validation Error	Return structured error to client	No retry	Client receives error response
Dataverse Throttling	Retry with backoff	5 retries	Admin alert if persistent
Network Timeout	Retry	3 retries	Log incident

Structured Error Response Format:

All broker errors return a consistent JSON envelope with a code, human-readable message, contextual details (envelope ID, status, relevant counts), and a UTC timestamp. This ensures client applications can handle errors programmatically rather than parsing free-text messages.

Part 3: Document Templating & Power Pages Portal

6. Feature Overview

6.1 Feature Purpose

The Document Templating feature extends the ESign Elections Canada Broker Service with a self-service capability that allows EC staff to independently prepare and send documents for e-signature, without requiring pre-built Nintex templates or IT involvement for each new document type.

6.2 Business Value

Benefit	Description	Impact
User Empowerment	EC staff create and submit their own templates	Reduced IT dependency
Flexibility	Supports both Word (.docx/.doc) and PDF templates	Broader use cases
Speed	No manual Nintex template configuration required	Faster time-to-signature
Consistency	Standardized placeholder-based field mapping	Fewer placement errors
Self-Service	Guided portal workflow	Lower support overhead

6.3 Feature Scope

In Scope:

- Power Pages portal for document upload and field mapping
- Entra ID SSO integration (session-based, no re-authentication if already signed in)
- Word-to-PDF conversion engine (LibreOffice Online)
- PDF field extraction and visual mapping interface
- Integration with the existing `cs_envelope` / `cs_SendEnvelope` workflow
- Template library management (save and reuse field mappings)

Out of Scope:

- Direct Nintex template creation (use the Nintex administration portal)
- Advanced PDF editing or annotation capabilities beyond field placement
- Batch document processing
- Mobile app development

6.4 Supported Document Types

Format	Extension	Conversion Required	Max Size	Notes
PDF	.pdf	No	10 MB	Native support
Word 2007+	.docx	Yes → PDF via LibreOffice	10 MB	Recommended format
Word 97-2003	.doc	Yes → PDF via LibreOffice	10 MB	Legacy support

6.5 Supported Field Types

Field Type	Nintex Equivalent	Description
Signature	Signature	Full handwritten/electronic signature capture
Initial	Initial	Initials only
Date	Date Signed	Auto-populated signing date
Text	Text Field	Free-form text entry
Checkbox	Checkbox	Boolean selection
Radio	Radio Button	Single selection from a group

6.6 Document Placeholder Syntax

EC staff mark their Word or PDF documents with placeholder text before uploading. The portal's field extraction engine detects these placeholders and automatically creates field mapping entries.

Placeholder Format: `{{TYPE:Label:SignerIndex}}`

Placeholder	Meaning
`{{SIGNATURE:1}}`	Signature field for Signer 1
`{{INITIAL:2}}`	Initials field for Signer 2
`{{DATE:1}}`	Date signed for Signer 1
`{{TEXT:Name:1}}`	Text field labelled "Name" for Signer 1
`{{CHECK:Agree:1}}`	Checkbox labelled "Agree" for Signer 1

Parsing rules: the format is case-insensitive; whitespace is trimmed; the Label component is optional for SIGNATURE, INITIAL, and DATE fields; SignerIndex defaults to 1 if omitted.

6.7 End-to-End User Workflow

Step 1 – Template Preparation (External) The user creates a Word or PDF document outside the portal, inserts placeholder text for each required signature field, and saves the file locally.

Step 2 – Portal Access The user clicks "Send for E-Signature" in their application. They are redirected to the Power Pages portal and authenticated automatically via Entra ID SSO if a valid session exists, or prompted for credentials and MFA if not.

Step 3 – Document Upload The user drags and drops or browses to upload their document. The portal validates the file type and size. If the document is a Word file, it is automatically converted to PDF via the LibreOffice Online service. The resulting PDF is stored in Dataverse.

Step 4 – Field Mapping The portal parses the PDF and extracts all placeholder text. The user is presented with a split-screen view: the PDF preview on the left with detected fields highlighted, and a field configuration panel on the right where each detected field is assigned a type and signer.

Step 5 – Envelope Configuration The user enters signer information (email, name, optional phone, and preferred language), sets envelope properties (subject, message, expiry in days, reminder frequency), and configures signing options (sequential vs. parallel, hide signer info, allow decline).

Step 6 – Preview and Submit The user reviews a summary of the envelope and confirms accuracy. Upon submission, the portal creates all required Dataverse records (`cs_envelope` , `cs_document` , `cs_field` , `cs_signer`) and calls the `cs_SendEnvelope` custom action. The existing Power Automate flow handles Nintex submission. Signers receive email invitations from Nintex.

7. Portal Architecture & Design

7.1 Component Architecture

The portal solution consists of the following components:

Power Pages Portal – The web interface layer, hosted on the Microsoft Power Platform (Canada Central). Responsible for authentication, document upload handling, and interaction with Dataverse via the Portal Web API.

Document Upload Handler – Client-side validation of file type and size, Base64 encoding, and routing to the conversion service for Word files.

PDF Conversion Service – LibreOffice Online deployed as an Azure Container Instance. Accepts Word documents and returns PDF output. API key is stored in Azure Key Vault.

Field Extraction Engine – Client-side PDF.js library combined with a custom JavaScript parser that scans page text content for placeholder patterns and extracts position coordinates.

Field Mapping UI – Interactive dual-panel interface showing the PDF preview and field configuration list.

Dataverse (Broker) – Backend data store shared with the broker service. The portal writes to the same `cs_envelope` , `cs_document` , `cs_field` , `cs_signer` , and `cs_template` tables used by programmatic clients.

7.2 Integration with the Existing Broker

The portal is designed as an additive client layer. It does not modify or replace the existing broker flows — it creates Dataverse records using the same schema and then triggers the same `cs_SendEnvelope` custom action that programmatic clients use.

The key difference is in how the Nintex payload is assembled downstream. The enhanced Power Automate flow (see Section 5.5) checks for `cs_field` records and, if present, includes them as explicit field position data in the Nintex submission rather than relying on a pre-configured Nintex template.

Existing Broker Flow: Client Application → Custom Connector → Entra ID → Dataverse → Power Automate → Nintex

New Templating Flow: Power Pages Portal → Entra ID (SSO) → Dataverse → (same Power Automate flow, enhanced) → Nintex

7.3 Portal Page Structure

The portal is organized as a four-step guided workflow:

Page	URL Path	Purpose
Document Upload	/upload	File selection, validation, and conversion
Field Mapping	/field-mapping	Placeholder detection and field assignment
Envelope Configuration	/envelope-config	Signer details and envelope properties
Preview & Submit	/preview	Final review and submission
Template Library	/my-templates	Save, manage, and reuse templates

Page	URL Path	Purpose
Help	/help	User guide and FAQ

7.4 Authentication Configuration

The portal uses the OpenID Connect protocol with the Elections Canada Entra ID tenant as the identity provider.

Configuration Item	Value
Provider Type	OpenID Connect (Entra ID External)
Authority	https://login.microsoftonline.com/{tenant-id}/v2.0
Client ID	[FILL – Power Pages App Registration]
Client Secret	[FILL – stored in Azure Key Vault]
Redirect URI	https://{portal-subdomain}.powerappsportals.com/signin-oidc
Scopes	openid, profile, email
Session Timeout	8 hours (matches Conditional Access policy)
Idle Timeout	2 hours
Claims Mapping	email → emailaddress1; name → fullname; oid → externalidentityid

7.5 Page Design Descriptions

Upload Page

The upload page presents a drag-and-drop file drop zone that accepts `.pdf`, `.docx`, and `.doc` files up to 10 MB. File type and size are validated on the client side before submission. An informational panel explains the supported placeholder syntax. After a valid file is selected and conversion (if required) completes successfully, the user proceeds to the field mapping page.

Field Mapping Page

The field mapping page displays a split-screen layout. The left panel renders the PDF using PDF.js and overlays coloured highlighting on each detected placeholder. The right panel lists each detected field with controls to assign its type and signer. Users may also manually add additional fields that were not defined via placeholders (for example, if a signer needs to initial at multiple locations). Each field card on the right is linked to its visual representation on the left to aid accurate positioning.

Envelope Configuration Page

This page collects signer information for each required signing party and envelope-level settings (subject, message to signers, expiry period, reminder frequency, sequential vs. parallel signing, signer visibility, and decline allowance). The number of signer slots is dynamically determined by the highest signer index found in the field mapping step.

Preview and Submit Page

The preview page summarizes all configured settings — document name, page count, detected field count, number of signers, signing mode — and displays a read-only PDF preview. The user must confirm accuracy via a checkbox before the “Send for E-Signature” button becomes active. A progress modal is displayed during the multi-step Dataverse record creation and `cs_SendEnvelope` call, providing real-time feedback at each step.

7.6 Portal Web API Integration

The portal interacts with Dataverse through the Power Pages Portal Web API (`/_api/`), which is the standard Dataverse OData interface scoped to the authenticated Contact's permissions. The portal creates records in this order: `cs_envelope` (Draft) → `cs_document` → `cs_signer` (one per signer) → `cs_field` (one per mapped field) → POST to `cs_SendEnvelope` .

All operations are owner-scoped; the portal automatically sets `ownerid` to the authenticated Contact, and Dataverse RLS prevents cross-user data access.

8. Document Processing Pipeline

8.1 Word-to-PDF Conversion

Word documents uploaded to the portal are converted to PDF before field extraction and storage. The recommended conversion mechanism is **LibreOffice Online (Collabora)** deployed as an Azure Container Instance in the same region as the broker environment (Canada Central).

Service Configuration:

Setting	Value
Deployment	Azure Container Instance
Image	collabora/code:latest
Resources	2 vCPU, 4 GB RAM
Region	Canada Central
API Authentication	API Key (stored in Azure Key Vault)
Endpoint	https://[FILL].azurecontainer.io

The portal sends the Base64-encoded Word document to the converter with options to preserve formatting, embed fonts, and output PDF version 1.7 (compatible with Nintex). The converter returns the Base64-encoded PDF along with page count, file size, and conversion duration metadata.

Conversion Quality Settings:

Setting	Value	Impact
DPI	150	Good quality with reasonable file size
Image Compression	Medium	Balanced quality and size
Font Embedding	All fonts	Ensures consistent rendering across viewers
PDF Version	1.7	Compatible with Nintex AssureSign

Fallback Option: An Azure Functions-based conversion endpoint using Aspose.Words is available as an alternative if the LibreOffice container is unavailable, providing redundancy for this pipeline stage.

8.2 PDF Field Extraction

Once a PDF is available (either uploaded directly or converted from Word), the portal's field extraction engine parses all pages for placeholder text matching the `{{TYPE:Label:Signer}}` pattern.

The extraction process uses PDF.js to access the text content layer of each page, iterates through text items matching the placeholder regular expression, and records each field's type, label, signer index, page number, and pixel coordinates (X, Y, width, height) based on the PDF text item's transform matrix.

Fields are returned to the field mapping UI as a structured list of field objects, each containing the information needed to both display the visual overlay on the PDF and populate the `cs_field` record in Dataverse upon submission.

8.3 Coordinate System Conversion

PDF documents use a coordinate system with the origin at the bottom-left corner of the page, with the Y-axis increasing upward. Nintex uses a coordinate system with the origin at the top-left corner, with the Y-axis increasing downward. The broker applies the conversion formula `nintexY = pageHeight - pdfY - fieldHeight` to all extracted field positions before including them in the Nintex submission payload.

8.4 Template Reuse

After successfully submitting an envelope via the portal, the user is offered the option to save the document and its field mappings as a named template. Saved templates are stored as `cs_template` records with associated `cs_field` records. When creating a future envelope from a saved template, the portal pre-populates the field mapping step with the saved positions, so the user only needs to enter the signer details before proceeding to submission.

Templates may be marked as public (`cs_ispublic = true`), making them available to all authenticated portal users as a shared starting point (e.g., standard contract templates for common use cases). Private templates are only visible to their creator.

9. Templating Security & Access Control

9.1 Portal Security Architecture

The portal's security model extends the broker's existing defence-in-depth approach with portal-specific controls:

Authentication – Entra ID SSO via OpenID Connect. If a session already exists, authentication is transparent to the user. MFA is enforced by the existing Conditional Access policy.

Authorization – Table permissions on the Power Pages portal define what authenticated Contacts can read and write. The Portal Template User security role in Dataverse enforces these permissions at the data layer.

Row-Level Security – All envelope, document, signer, and field records created through the portal are owned by the authenticated Contact. RLS prevents any portal user from accessing another user's records.

Content Security – HTTPS-only access is enforced, secure cookies are used for session management, and a Content Security Policy (CSP) header restricts script execution to trusted sources.

9.2 Web Role Configuration

Setting	Value
Web Role Name	Authenticated Users – Template Portal
Assignment	Automatically assigned to all authenticated Contacts on first login
Parent Role	None
Content Management	No

Table Permissions:

Table	Access Type	Privileges
cs_template (read)	Global (all records)	Read
cs_template (manage own)	Contact (ownerid = current Contact)	Create, Read, Write, Delete
cs_envelope	Contact (ownerid = current Contact)	Create, Read, Write
cs_document	Parental (via cs_envelope)	Create, Read, Append To
cs_field	Parental (via cs_envelope)	Create, Read, Append To
cs_signer	Parental (via cs_envelope)	Create, Read, Append To

9.3 Data Security Controls

Control	Implementation	Purpose
Authentication	Entra ID SSO (OIDC)	Identity verification
Authorization	Table permissions (owner-based)	Access control
Row-Level Security	ownerid field filtering	Multi-user data isolation
Encryption at Rest	Dataverse TDE (AES-256)	Data protection
Encryption in Transit	TLS 1.2+ (HTTPS)	Network security
Input Validation	Client-side and server-side	Prevent injection attacks
File Type Validation	Whitelist (.pdf, .docx, .doc only)	Malware surface reduction
File Size Limit	10 MB maximum	Denial-of-service prevention
Rate Limiting	Power Pages throttling	Abuse prevention
Audit Logging	Portal logs and Dataverse audit	Compliance and forensic trail

9.4 Threat Mitigation

Threat	Mitigation	Residual Risk
Unauthorized Access	Entra ID MFA, Conditional Access	Low
File Upload Malware	File type whitelist, size limits; antivirus scanning (planned)	Medium (pending antivirus integration)
Cross-Site Scripting	Input sanitization, CSP headers	Low
CSRF	Anti-forgery tokens (Power Pages built-in)	Low
SQL/OData Injection	Dataverse Web API parameterized queries	Low
Session Hijacking	Secure cookies, HTTPS only, session timeout	Low
Data Exfiltration	RLS, table permissions, audit logging	Low
Denial of Service via Large Files	10 MB file size limit, rate limiting	Low

10. Templating Data Model Extensions

10.1 Summary of Changes

Version 2.0 introduces one new Dataverse table (`cs_template`) and extends two existing tables (`cs_field` with new columns; `cs_envelope` with an optional template reference). All changes are backward-compatible; existing programmatic clients are unaffected by the additions.

10.2 New Table: cs_template

See Section 4.2.5 for the full field schema. Key design considerations:

The `cs_ispublic` flag allows the platform team to designate canonical templates (e.g., standard government form templates) that all portal users can discover and use. Creators retain write access to their own templates regardless of the public flag.

The `cs_fieldcount` field is a calculated integer maintained by the portal during template creation. It provides a quick summary of field complexity without requiring an expand query on `cs_field` .

Template deletion cascades to associated `cs_field` records (template-linked fields only). Envelope-linked `cs_field` records are unaffected, as they may reference a template by ID as a provenance indicator but are not dependent on the template record for runtime operation.

10.3 Modified Table: cs_field

Five new columns are added to support the richer field model required by the portal:

New Field	Type	Purpose
cs_templateid	Lookup → cs_template	Links field to its source template (optional)
cs_fieldlabel	String (100)	Human-readable display label shown to the signer
cs_isrequired	Boolean (default: true)	Whether the signer must fill this field to complete signing
cs_dateformat	String (20)	Date format string for date fields (e.g., “MM/dd/yyyy”)
cs_maxlength	Integer	Maximum character length for text fields

10.4 Updated ERD Relationships

The v2.0 data model adds the following relationships to the existing ERD:

- `cs_template` → `cs_field` (1:N, cascade delete, relationship name: `cs_template_field`)
- `cs_envelope` → `cs_template` (N:1 optional, tracking provenance of portal-originated envelopes)
- Existing relationships (`cs_envelope` → `cs_field` , `cs_signer` → `cs_field`) are unchanged.

11. Templating Implementation & Deployment

11.1 Prerequisites

Azure Infrastructure:

Resource	Purpose	Configuration
Power Pages Portal	Self-service web interface	Entra ID authentication enabled; connected to broker Dataverse
Azure Container Instance	LibreOffice Word-to-PDF converter	Linux; 2 vCPU; 4 GB RAM; Canada Central
Azure Key Vault (existing)	Store LibreOffice API key and portal client secret	Add new secrets to existing broker Key Vault
Application Insights (existing)	Monitor portal and conversion service	Add portal instrumentation key

Licenses:

License	Quantity	Purpose
Power Pages (Authenticated Users)	Per authenticated user	Portal access
Dataverse Storage (incremental)	As needed	Document and template storage
Azure Container Instance	1 instance (prod), 1 instance (dev)	PDF conversion

11.2 Implementation Phases

Phase	Activities	Duration
Phase 1 – Infrastructure Setup	Create Power Pages portal; configure Entra ID provider; deploy LibreOffice ACI; set up Application Insights	Week 1
Phase 2 – Dataverse Extensions	Create cs_template table; modify cs_field table; configure security roles; create table permissions for portal	Week 1
Phase 3 – Portal Development	Build upload page; implement field extraction; create field mapping UI; build envelope configuration and preview pages	Weeks 2–3
Phase 4 – Integration	Connect conversion service; wire Dataverse Web API calls; modify Power Automate flow (enhanced payload); end-to-end testing	Week 4
Phase 5 – Testing	Unit, integration, UAT, performance, and security testing	Week 5
Phase 6 – Deployment & Training	Deploy to production; user training; go-live support; documentation	Week 6

11.3 Deployment Checklist

Pre-Deployment:

- ☐ Backup existing Dataverse environment
- ☐ Test rollback procedure in non-production environment
- ☐ Verify all prerequisites are met (licenses, infrastructure)
- ☐ Obtain Change Advisory Board (CAB) approval

Deployment Steps:

1. Import the `ESignTemplatingFeature` Dataverse solution (contains `cs_template` table, `cs_field` modifications, new security role, and portal web resources)

2. Configure Power Pages portal authentication (Entra ID OpenID Connect provider, web roles, table permissions, CSP headers)
3. Upload portal web files (HTML pages, CSS, JavaScript libraries including PDF.js)
4. Deploy LibreOffice Azure Container Instance
5. Import modified Power Automate flow solution (enhanced `cs_SendEnvelope` flow)
6. Configure Application Insights monitoring and alerts

Post-Deployment Validation:

- ☐ Smoke test all portal pages and navigation
- ☐ Test Word document upload and conversion
- ☐ Test PDF upload and field extraction
- ☐ Verify end-to-end envelope submission via portal
- ☐ Confirm signers receive email invitations
- ☐ Verify RLS (portal user cannot access another user's envelopes)
- ☐ Monitor Application Insights and Dataverse audit logs for the first 24 hours

11.4 Rollback Procedure

If the deployment fails or critical issues are identified post-deployment:

1. Delete the `ESignTemplatingFeature` Dataverse solution (reverts `cs_template` table creation and `cs_field` modifications)
2. Restore from the pre-deployment Dataverse backup
3. Delete the LibreOffice Azure Container Instance
4. Revert the Power Automate flow to the previous version
5. Remove the Power Pages portal or disable authentication
6. Document the incident and update the deployment plan before reattempting

11.5 Testing & Validation

Key Test Scenarios

Scenario	Steps	Success Criteria
Upload Word Document	Upload .docx with placeholders; verify PDF conversion	Conversion in <10 sec; all placeholders detected; field positions accurate to ±5px
Upload PDF Document	Upload .pdf with placeholders; verify field extraction	Fields detected without conversion step
Map Fields to Signers	Assign detected fields to signer types	Field assignments saved; PDF preview shows highlights
Create and Send Envelope	Configure envelope; add 2 signers; submit	Envelope status = Submitted; signers receive email within 5 minutes
Reuse Template	Save template; create new envelope from template	Pre-populated field positions; only signer details required
RLS Isolation	Attempt to access another user's envelope	404 or 403 response; no data leakage

Performance Benchmarks

Operation	Target	Acceptable	Unacceptable
Portal Page Load	<2 sec	<5 sec	>5 sec

Operation	Target	Acceptable	Unacceptable
Word-to-PDF Conversion	<10 sec	<20 sec	>20 sec
Field Extraction	<5 sec	<10 sec	>10 sec
Envelope Creation (all records)	<3 sec	<10 sec	>10 sec
End-to-End (Upload to Send)	<30 sec	<60 sec	>60 sec

User Acceptance Testing

UAT will be conducted with 5 EC staff from different departments, 1 IT administrator, and 1 security reviewer. The UAT battery covers uploading employment contracts and vendor agreements, saving and reusing templates, and handling error conditions (invalid file type, missing fields, oversized file).

Success Criteria: 90%+ task completion rate; average user satisfaction ≥ 4/5; fewer than 5 support tickets in the first week; no critical defects.

11.6 Operations & Support

Monitoring Alerts

Alert	Condition	Action
High Conversion Failures	More than 10 failures in 1 hour	Email IT team
Slow Conversion	P95 latency > 30 seconds	Email platform team
Converter Unavailable	Container health check failure	Page on-call engineer
Storage Capacity	Dataverse usage > 80%	Email admin team

Common Issues

Conversion Fails ("Document conversion failed" error) Cause: LibreOffice container crashed or network timeout. Resolution: Restart the Azure Container Instance and review container logs. If recurring, check ACI resource limits and consider scaling up.

Fields Not Detected (placeholders visible but no fields appear) Cause: Incorrect placeholder syntax or encoding issue. Resolution: Verify the placeholder format is exactly `{{TYPE:Label:Signer}}`, ensure the document uses UTF-8 encoding, check for hidden characters within placeholders, re-save the Word document and retry.

Incorrect Field Positions (fields appear in wrong location on PDF) Cause: Coordinate conversion error or non-standard font metrics. Resolution: Use standard fonts (Arial, Times New Roman); avoid complex nested layouts; use the manual position adjustment controls in the field mapping UI.

Support Information

Issue Type	Contact	Response Time
Portal Login	IT Helpdesk	1 hour
Upload or Conversion Error	Platform Team	4 hours
Field Mapping Issues	Platform Team	4 hours
Template Questions	User Guide and FAQ	Self-service
Critical Production Bug	On-call Engineer	15 minutes

Part 4: Security & Monitoring

12. Security Architecture

12.1 Defence in Depth

The platform applies security controls across seven layers:

Layer 1 – Compliance: Audit logging for all actions; ITSG-33 control alignment; PII management.

Layer 2 – Secrets: Azure Key Vault for all credential storage; Managed Identities to eliminate secrets in code; quarterly secret rotation.

Layer 3 – Identity: Entra ID authentication; Conditional Access policy enforcement; MFA for administrator accounts.

Layer 4 – Network: TLS 1.2+ encryption in transit; optional IP restrictions; Azure Front Door DDoS protection.

Layer 5 – API Gateway: OAuth 2.0 token validation; per-client API throttling; full request logging.

Layer 6 – Application: Input validation (XSS and injection prevention); custom action business logic security; DLP policies; portal-specific file type and size controls *(v2.0)*.

Layer 7 – Data: AES-256 encryption at rest; Column-Level Security on sensitive fields; Row-Level Security for per-client and per-user data isolation.

12.2 Threat Model – STRIDE Analysis

Threat	Attack Vector	Mitigation	Residual Risk
Spoofing	Impersonate client application or service	OAuth 2.0, service principals, Entra ID SSO	Low
Tampering	Modify envelope data	RLS, audit logging, immutable logs	Low
Repudiation	Deny sending an envelope	Complete audit trail in cs_apirequest	Low
Information Disclosure	Access another application's data	RLS, CLS, encryption	Low
Denial of Service	Flood API or portal with requests	API throttling, DLP policies, portal file size limits	Medium
Elevation of Privilege	Gain admin access	Least privilege, MFA, PIM	Low

12.3 Data Classification

Data Element	Classification	Encryption	Access Control	Retention
Envelope metadata (name, subject)	Protected B	At rest and in transit	RLS	7 years

Data Element	Classification	Encryption	Access Control	Retention
Signer PII (email, name, phone)	Protected B	At rest and in transit	RLS + CLS	7 years
Document content (Base64)	Protected B	At rest and in transit	RLS	7 years
Signing links	Protected B	At rest and in transit	RLS + CLS	Until expiry
API request/response logs	Protected B	At rest and in transit	Admin only	7 years
Nintex API credentials	Secret	Azure Key Vault	MSI only	Rotate quarterly
Client secrets	Secret	Azure Key Vault	Admin only	2-year maximum

12.4 PII Management

PII Elements Stored:

Element	Table	Purpose	Legal Basis
Email address	cs_signer	Contact signer	Consent via envelope submission
Full name	cs_signer	Display on document	Consent via envelope submission
Phone number	cs_signer	Optional contact method	Consent via envelope submission
IP address	cs_signer	Audit trail	Legitimate interest (fraud prevention)

Data Subject Rights:

Right	Implementation	Response Time
Access	Export via Dataverse API	30 days
Rectification	Update signer record	Immediate
Erasure	Delete envelope (cascade)	30 days
Portability	Export as JSON/CSV	30 days
Objection	Opt-out via configuration	Immediate

13. Monitoring, Logging & Audit

13.1 Monitoring Architecture

All logging sources flow into a central Log Analytics workspace (`broker-prod-logs` , 2-year retention). The primary data sources are Dataverse Audit Logs, Power Automate Run History, Azure Monitor Metrics, Application Insights (used by the portal and conversion service in v2.0), Key Vault Logs, and Entra ID Sign-in Logs.

Azure Monitor Dashboards provide three operational views: Operational Health (API success rates, latency, envelope volume), Security Monitoring (failed authentications, unusual IP addresses, privileged access events), and Client Usage (top clients by volume, completion rates, failed envelopes by client).

Alerts are exported to email and Microsoft Teams notifications. SIEM export is available optionally.

13.2 Logging Strategy

All Nintex API calls are logged to `cs_apirequest` records with the full request body, response body, HTTP status code, success flag, timestamp, and duration. This provides a complete Protected B-compliant audit trail for all Nintex interactions.

In v2.0, portal activity is additionally logged to Application Insights via custom events and metrics, capturing document upload outcomes, conversion performance, field detection counts, and any client-side errors.

Retention: 7 years for Dataverse audit data and cs_apirequest records (Protected B requirement).

13.3 Alerting Rules

Critical Alerts (Immediate Response):

Alert	Condition	Threshold	Action
Nintex API Failure	HTTP 5xx errors	>5 in 5 minutes	Page on-call engineer
Authentication Failure	401/403 errors	>10 in 5 minutes	Security team notification
Data Breach Attempt	Cross-tenant access attempt	Any	Immediate investigation
Service Degradation	API latency > 5 seconds	95th percentile	Incident response

Warning Alerts (Business Hours):

Alert	Condition	Threshold	Action
Token Expiry	Secret expires in <60 days	N/A	Rotation reminder
High Error Rate	Failed flows	>10% in 1 hour	Investigate root cause
Storage Capacity	Dataverse usage	>80%	Capacity planning
Client Quota Exceeded	Monthly envelope volume	>Limit	Billing notification
Conversion Failures*(v2.0)*	Portal Word-to-PDF failures	>10 in 1 hour	Email platform team
Converter Unavailable*(v2.0)*	LibreOffice ACI health check failure	Any	Page on-call

13.4 Audit Trail Requirements (ITSG-33)

All CRUD operations on Protected B data entities (`cs_envelope` , `cs_signer` , `cs_document` , `cs_field` , `cs_apirequest`) generate Dataverse audit log entries capturing the timestamp, user ID, operation type, entity name, and record ID. Service principal creation and deletion events are captured in Entra ID logs and can be queried via Log Analytics.

13.5 Log Retention

Log Type	Retention Period	Archive Location	Legal Hold
Dataverse Audit Logs	7 years	Azure Storage (Cool tier)	Available
cs_apirequest	7 years	In-place (Dataverse)	Available
Azure Monitor Logs	2 years	Log Analytics	Available
Power Automate Run History	28 days	Power Platform	N/A
Entra ID Sign-in Logs	1 year	Azure AD Premium	Available

Log Type	Retention Period	Archive Location	Legal Hold
Key Vault Access Logs	2 years	Storage Account	Available
Application Insights*(v2.0)*	90 days	Application Insights	N/A

Part 5: Operations & Compliance

14. Data Retention & Disposition

14.1 Retention Requirements (Protected B)

All Protected B data must be retained for a minimum of 7 years in accordance with Government of Canada standards.

14.2 Retention Policies

Dataverse Data:

Entity	Retention Period	Disposition Method	Trigger
cs_envelope	7 years from completion	Hard delete	Automated daily job
cs_signer	7 years from completion	Hard delete (cascade)	Parent envelope deletion
cs_document	7 years from completion	Hard delete (cascade)	Parent envelope deletion
cs_apirequest	7 years from creation	Hard delete	Automated daily job
cs_template	Active templates only	Soft delete (deactivate)	Manual

The automated retention job runs daily at 2:00 AM. It exports envelopes past their 7-year mark to Azure Blob Storage (immutable, Cool tier) before deletion, then hard-deletes the Dataverse records in cascade order. API request records older than 7 years are deleted in batches of 1,000 to avoid throttling.

Nintex AssureSign Data:

Data Type	Nintex Retention	Disposition	Notes
Envelopes	7 years	Auto-delete by Nintex	Configurable in Nintex admin
Signed Documents	7 years	Auto-delete by Nintex	Downloadable before deletion
Audit Trails	7 years	Auto-delete by Nintex	Export available

Before Nintex auto-deletion, the broker downloads signed documents via the `/getCompletedDocument` endpoint and stores them in Azure Blob Storage (immutable) as the authoritative archive copy.

14.3 Backup Strategy

Component	Backup Frequency	Retention	RPO
Dataverse (continuous)	Continuous	28 days	<1 hour

Component	Backup Frequency	Retention	RPO
Dataverse (manual weekly)	Weekly	90 days	<24 hours
Key Vault	Continuous	90 days (soft-delete)	<1 hour

RTO: 4 hours | RPO: 24 hours

14.4 Legal Hold Process

When a legal hold is requested, the scope is defined in collaboration with Legal, affected records are identified and tagged with a legal hold flag, automated deletion is suspended for those records, and an export to immutable Azure Blob Storage is made. Chain of custody is documented. Upon hold release (following quarterly review), the legal hold flag is removed and normal retention resumes.

15. Client Onboarding Process

15.1 Onboarding Workflow

Client onboarding follows a structured lifecycle: intake and requirements gathering → service principal creation and Dataverse configuration → credential packaging and secure delivery → training and integration support → testing → production go-live → ongoing monitoring and quarterly review.

15.2 Onboarding Timeline

Total typical timeline: **14–21 business days.**

Phase	Activities	Duration
Intake	Requirements gathering, intake form completion	Day 1
Provisioning	Create service principal, configure Dataverse application user and security role	Days 1–2
Delivery	Package credentials, deliver via secure channel, provide integration guide	Days 2–3
Training	Schedule and conduct training session	Days 3–5
Testing	Client tests integration; issue resolution	Days 7–14
Go-Live	Production approval, cutover, enhanced monitoring	Days 14–21
Review	Week 1 post-go-live review, Week 4 review	Ongoing

15.3 Onboarding Checklist

Phase 1: Intake (Day 1)

Field	Value
App/Service Name	[FILL]
Primary Contact	[FILL]
Email	[FILL]

Field	Value
Phone	[FILL]
Dataverse Environment URL	[FILL]
Expected Monthly Volume	[FILL]
Use Cases	[FILL]
Security Clearance Level	[FILL]
Approval Required?	☐ Yes ☐ No

Phase 2: Service Principal Creation (Day 1–2)

Steps performed by the Security Lead:

1. Create App Registration in Entra ID (`ESign-{ServiceName}-Prod`)
2. Create Service Principal from App Registration
3. Generate client secret with 2-year expiry
4. Grant Dataverse API permissions and obtain admin consent
5. Store client secret in Azure Key Vault

Phase 3: Dataverse Configuration (Day 2)

Setting	Value
Application User ID	[FILL]
Service Principal Object ID	[FILL]
Security Role	Client Access
Business Unit	Root
Created Date	[FILL]
Created By	[FILL]

Phase 4: Testing (Days 7–14)

Test Case	Description	Expected Result	Status
TC-001	Import custom connector	Success	☐
TC-002	Create connection	Success	☐
TC-003	Create draft envelope	Envelope created, status = Draft	☐
TC-004	Add signer	Signer added	☐
TC-005	Add document	Document attached	☐
TC-006	Send envelope	Status = Submitted or Pending Approval	☐
TC-007	Get envelope details	Data returned	☐
TC-008	List envelopes	Only client’s envelopes returned	☐
TC-009	Per-client isolation	Cannot access other client data	☐
TC-010	Error handling	Graceful error messages	☐

15.4 Onboarding Roles & Responsibilities

Role	Responsibilities	Contact
Platform Lead	Overall ownership and approvals	[FILL]
Security Lead	Service principal creation and permissions	[FILL]
Technical Lead	Dataverse configuration and training	[FILL]
Support Lead	Documentation and ongoing support	[FILL]
Client Project Manager	Client-side coordination	[FILL: Client]
Client Technical Lead	Integration development	[FILL: Client]

16. Environment Strategy

16.1 Environment Architecture

The platform maintains four Dataverse environments aligned to the standard software development lifecycle:

Development (`dev-broker-esign` , Canada Central) – Feature development and unit testing. Uses a Nintex sandbox account. Developer-tier Dataverse. No DLP policy applied.

QA (`qa-broker-esign` , Canada Central) – User acceptance testing and integration testing. Uses a Nintex sandbox account. Production-tier Dataverse (2 GB base).

Production (`prod-broker-esign` , Canada Central) – Live client workloads. Uses the Nintex production account. Production-tier Dataverse (10 GB base + usage-based).

Disaster Recovery (`dr-broker` , Canada East) – Warm standby in a separate Azure region. Maintains a current backup of the production environment. Activates via the documented DR procedure (see Section 18).

16.2 DLP Policies

The global DLP policy (`Elections Canada – Broker Service Global`) applies to all environments except Development:

Allowed Connectors: Dataverse (broker environment only), Azure Key Vault, Office 365 Outlook, Approvals, Custom Connector “ESign Elections Canada” (wildcard: *ESign*).

Blocked Connectors: All other connectors by default.

Rules: No data exfiltration to consumer services (e.g., Gmail, Dropbox); no cross-environment data flows; all API calls must be audited.

17. Network Architecture

17.1 Network Topology

All inbound traffic from client applications and portal users passes through Azure Front Door (WAF and DDoS protection) before reaching the Power Platform endpoints over TLS 1.2+. The broker’s Power Automate flows communicate outbound to Nintex AssureSign (`api.assuresign.net`) over HTTPS.

Azure services in Canada Central (Key Vault via Private Endpoint, Azure Monitor via Service Endpoint, Azure Storage via Private Endpoint) are accessible to Power Automate flows through managed connectors.

The v2.0 LibreOffice Azure Container Instance is accessible to the Power Pages portal over HTTPS using an API key. It has no public inbound access outside of the portal's conversion requests.

17.2 Latency Targets

Endpoint	Target	Measurement Method
Client/Portal → Dataverse API	<200 ms	Azure Monitor
Dataverse → Nintex API	<500 ms	Application Insights
End-to-End (Submit Envelope)	<3 seconds	Flow analytics
Portal Word-to-PDF Conversion*(v2.0)*	<10 seconds	Application Insights

18. Disaster Recovery & Business Continuity

18.1 Recovery Objectives

Metric	Target	Maximum Tolerable
RTO (Recovery Time Objective)	4 hours	8 hours
RPO (Recovery Point Objective)	24 hours	48 hours
MTTR (Mean Time To Repair)	2 hours	4 hours
MTBF (Mean Time Between Failures)	720 hours (30 days)	N/A

18.2 Disaster Scenarios

Scenario	Type	Response
Regional Outage (Canada Central)	Infrastructure	Activate DR environment in Canada East
Dataverse Corruption / Data Loss	Data	Point-in-time restore from continuous backup
Security Breach	Security	Incident response and containment procedure
Nintex Outage	External dependency	Wait for Nintex recovery; queue submissions
LibreOffice Converter Outage*(v2.0)*	Infrastructure	Restart ACI; fallback to Azure Functions converter if persistent

18.3 Regional Outage Recovery Procedure (Hour-by-Hour)

Hour 0–1: Assessment Verify outage scope via Azure Status page. Test broker API endpoint. Notify stakeholders with ETA of 4 hours.

Hour 1–2: Activation Restore latest Dataverse backup to DR environment (Canada East). Monitor restore progress.

Hour 2–3: Configuration Import Power Automate solutions from Git repository. Reconfigure Managed Identity access to Key Vault. Update DLP policies to include DR environment. Test Nintex integration from DR environment.

Hour 3–4: Validation and Communication Submit 3 test envelopes end-to-end. Verify status sync. Send notification to all clients with the new temporary endpoint URL.

18.4 Security Breach Response

Upon breach detection: immediately revoke all client secrets and disable compromised accounts; enable IP restrictions to known GC IP ranges. Conduct forensic analysis to identify scope. If PII data was exfiltrated, invoke the Privacy Breach Protocol and notify the Privacy Commissioner. Remediate the vulnerability, issue new credentials, and apply enhanced monitoring for 30 days. Conclude with a post-incident review and security control updates.

Incident Response Contacts:

Role	Name	Phone	Email
Incident Commander	[FILL]	[FILL]	[FILL]
Security Lead	[FILL]	[FILL]	[FILL]
Privacy Officer	[FILL]	[FILL]	[FILL]
Communications Lead	[FILL]	[FILL]	[FILL]

18.5 Critical Business Functions

Function	RTO	RPO	Dependencies
Create Envelope	4h	24h	Dataverse, Entra ID
Send Envelope	4h	24h	Dataverse, Nintex, Key Vault
Status Sync	8h	48h	Nintex API
Portal Upload*(v2.0)*	4h	24h	Dataverse, Entra ID, LibreOffice ACI
Client Onboarding	24h	N/A	Entra ID (manual process fallback available)

19. Compliance & Governance

19.1 ITSG-33 Control Mapping

Access Control (AC):

Control	Requirement	Implementation
AC-1	Access control policy	Security roles, RLS, OAuth 2.0
AC-2	Account management	Service principals per client; quarterly review
AC-3	Access enforcement	Dataverse RLS, security roles
AC-4	Information flow enforcement	DLP policies, network isolation
AC-6	Least privilege	Minimal permissions per role
AC-7	Unsuccessful logon attempts	Entra ID lockout after 5 failures
AC-17	Remote access	VPN/Conditional Access required

Control	Requirement	Implementation
AC-20	Use of external systems	Nintex via API only; portal via authenticated HTTPS only

Audit and Accountability (AU):

Control	Requirement	Implementation
AU-2	Audit events	All CRUD on Protected B data
AU-3	Content of audit records	Timestamp, user, action, result
AU-6	Audit review	Weekly security team review
AU-9	Protection of audit information	Immutable storage, RBAC
AU-11	Audit record retention	7 years for Protected B
AU-12	Audit generation	Dataverse, Azure Monitor, Entra ID

System and Communications Protection (SC):

Control	Requirement	Implementation
SC-7	Boundary protection	DLP policies, TLS 1.2+
SC-8	Transmission confidentiality	TLS 1.2+ for all communications
SC-12	Cryptographic key management	Azure Key Vault, quarterly rotation
SC-13	Cryptographic protection	AES-256 at rest, TLS 1.2+ in transit
SC-28	Protection of information at rest	Dataverse TDE, Storage encryption

19.2 Privacy Impact Assessment (PIA)

PII Collected:

PII Element	Purpose	Legal Authority	Retention
Signer email	Contact for signature request	Consent via envelope submission	7 years
Signer name	Display on document	Consent via envelope submission	7 years
Signer phone	Optional contact method	Consent via envelope submission	7 years
IP address	Audit trail (fraud prevention)	Legitimate interest	7 years

Privacy Controls Applied: Collection limitation; data quality; purpose specification; use limitation; security safeguards; openness (privacy policy published); individual participation (data subject rights supported); accountability (Privacy Officer designated).

19.3 Compliance Summary

Control Family	Total Controls	Implemented	Not Applicable
AC: Access Control	25	20	5
AU: Audit and Accountability	16	16	0
CM: Configuration Management	11	11	0

Control Family	Total Controls	Implemented	Not Applicable
CP: Contingency Planning	13	13	0
IA: Identification and Authentication	11	10	1
SC: System and Communications Protection	44	38	6
SI: System and Information Integrity	23	20	3
TOTAL	**143**	**128**	**15**

Overall Compliance Rate: 90% (128/143 controls implemented)

Part 6: Annexes

Annex A: Environment Configuration

A.1 Environment URLs

Environment	URL	Purpose
Development	https://[FILL].crm3.dynamics.com	Feature development
QA	https://[FILL].crm3.dynamics.com	Testing
Production	https://[FILL].crm3.dynamics.com	Live workloads
DR	https://[FILL].crm4.dynamics.com	Disaster recovery

A.2 Azure Resources

Resource Type	Name	Purpose	Resource Group
Key Vault (Dev)	[FILL]-dev-kv	Dev secrets	[FILL]
Key Vault (QA)	[FILL]-qa-kv	QA secrets	[FILL]
Key Vault (Prod)	[FILL]-prod-kv	Production secrets	[FILL]
Storage Account	[FILL]auditarchive	Audit log archive	[FILL]
Log Analytics	broker-prod-logs	Monitoring	[FILL]
Container Instance (Prod)*(v2.0)*	libreoffice-converter-prod	Word-to-PDF conversion	[FILL]
Container Instance (Dev)*(v2.0)*	libreoffice-converter-dev	Word-to-PDF conversion	[FILL]
Application Insights*(v2.0)*	[FILL]-appi	Portal and converter monitoring	[FILL]

A.3 Service Principal Object IDs

Client Application	Environment	Object ID	Created Date
[Client 1]	Production	[FILL]	[FILL]
[Client 2]	Production	[FILL]	[FILL]
[Client 3]	Production	[FILL]	[FILL]

A.4 Nintex Configuration

Environment	Account Type	API Endpoint	Credentials (Key Vault)
Development	Sandbox	https://sandbox.assuresign.net/v3.7	dev-brokerkv/NintexAPIKey
QA	Sandbox	https://sandbox.assuresign.net/v3.7	qa-brokerkv/NintexAPIKey
Production	Production	https://api.assuresign.net/v3.7	prod-brokerkv/NintexAPIKey

A.5 Production Client Inventory

App/Service	Service Principal	Onboarding Date	Monthly Volume
[FILL]	[FILL]	[FILL]	[FILL]
[FILL]	[FILL]	[FILL]	[FILL]

Annex B: Security Controls Mapping (ITSG-33)

See Section 19.1 for detailed ITSG-33 control mapping tables. For the full security controls workbook including evidence references, test results, and remediation status for all 143 controls, refer to the separate Security Controls Workbook maintained by the Security Lead.

Annex C: API Specifications

C.1 Custom Action: cs_SendEnvelope

Endpoint: `POST /cs_envelopes({envelopeId})/Microsoft.Dynamics.CRM.cs_SendEnvelope`

Request Body:

Field	Type	Required	Description
EnvelopeId	GUID	Yes	Identifier of the envelope to send

Success Response:

Field	Type	Description
Status	String	“Submitted”
Message	String	Human-readable confirmation (e.g., “Envelope sent to 2 signers”)
NintexEnvelopeId	String	Nintex-assigned envelope identifier

Error Response Structure:

The error response contains a code (machine-readable error identifier), a message (human-readable description), a details object (envelope ID, current status, relevant counts), and a UTC timestamp.

C.2 OData Query Examples

Get envelopes created today:

```
GET /api/data/v9.2/cs_envelopes?$filter=Microsoft.Dynamics.CRM.Today(PropertyName='createdon')&$select=cs_name,cs_status
```

Get envelope with signers expanded:

```
GET /api/data/v9.2/cs_envelopes(guid)?$expand=cs_envelope_signer($select=cs_email,cs_fullname,cs_signerstatus)
```

Count envelopes grouped by status:

```
GET /api/data/v9.2/cs_envelopes?$apply=groupby((cs_status),aggregate($count as total))
```

Get all draft envelopes (with RLS applied automatically):

```
GET /api/data/v9.2/cs_envelopes?$filter=cs_status eq 'Draft'&$select=cs_name,cs_subject,cs_status
```

C.3 Nintex API Endpoint Reference

Endpoint	Method	Purpose	Request Body
/authentication/apiUser	POST	Obtain access token	{APIUsername, Key, ContextUsername}
/submit	POST	Create and send envelope	Full envelope payload (see Section 5.4)
/get	GET	Get envelope details	Query parameter: envelopeId={id}
/getSigningLinks	GET	Retrieve per-signer signing URLs	Query parameter: envelopeId={id}
/cancel	POST	Cancel envelope	{EnvelopeID}
/listTemplates	GET	List available templates	None
/getCompletedDocument	GET	Download signed PDF	Query parameter: envelopeId={id}

Annex D: Power Pages Portal Configuration

D.1 Portal Settings

Setting	Value
Portal Name	ESign Template Portal
URL	https://[FILL].powerappsportals.com
Dataverse Environment	prod-broker-esign

Setting	Value
Primary Language	English
Secondary Language	Français
HTTPS Only	true
Secure Cookies	true
Max Upload Size	10 MB
Session Timeout	8 hours
Idle Timeout	2 hours

D.2 Authentication Configuration

Setting	Value
Provider Type	OpenID Connect (Entra ID)
Auto-Registration	Enabled
Email Claim Mapping	emailaddress1
Name Claim Mapping	fullname
OID Claim Mapping	externalidentityid
Client ID	[FILL – Power Pages App Registration]
Redirect URI	https://[FILL].powerappsportals.com/signin-oidc

D.3 Table Permissions Summary

Table	Web Role	Access Type	Privileges
cs_template (public read)	Authenticated Users – Template	Global	Read
cs_template (own manage)	Authenticated Users – Template	Contact	Create, Read, Write, Delete
cs_envelope	Authenticated Users – Template	Contact	Create, Read, Write
cs_document	Authenticated Users – Template	Parental (via cs_envelope)	Create, Read, Append To
cs_field	Authenticated Users – Template	Parental (via cs_envelope)	Create, Read, Append To
cs_signer	Authenticated Users – Template	Parental (via cs_envelope)	Create, Read, Append To

Annex E: Document Conversion Specifications

E.1 LibreOffice Online API

Endpoint: `POST /convert`

The request includes the Base64-encoded source document, source format (`docx` or `doc`), target format (`pdf`), and conversion options (preserve formatting, embed all fonts, PDF version 1.7, medium image compression).

The response includes the Base64-encoded PDF, page count, file size in bytes, and conversion duration in seconds.

Supported Input Formats:

Format	Extension	Notes
Word 2007+	.docx	Full support
Word 97-2003	.doc	Legacy support
OpenDocument Text	.odt	Full support
Rich Text Format	.rtf	Limited formatting support
Plain Text	.txt	Basic conversion

E.2 Conversion Quality Settings

Setting	Value	Purpose
DPI	150	Good quality with reasonable file size
Image Compression	Medium	Balanced quality and size
Font Embedding	All fonts	Consistent rendering
PDF Version	1.7	Nintex AssureSign compatibility

Annex F: Field Mapping Schema

F.1 Field Type Definitions

Each supported field type maps to a Nintex field type and has default dimensions:

Field Type	Nintex Type	Default Width (px)	Default Height (px)	Required Properties
Signature	Signature	200	50	signerIndex, pageNumber, x, y
Initial	Initial	100	40	signerIndex, pageNumber, x, y
Date	DateSigned	150	30	signerIndex, pageNumber, x, y
Text	TextField	200	30	signerIndex, pageNumber, x, y
Checkbox	Checkbox	20	20	signerIndex, pageNumber, x, y

F.2 Coordinate System

PDF documents use a bottom-left origin coordinate system (Y increases upward). Nintex uses a top-left origin coordinate system (Y increases downward). The broker applies the following conversion: `nintexY = pageHeight - pdfY - fieldHeight` .

This conversion is applied automatically during the `cs_SendEnvelope` flow when building the Nintex submission payload from `cs_field` records.

Annex G: Troubleshooting Guide

G.1 Common Issues (Broker Service)

Issue: "Unauthorized" when calling the API Cause: Token is invalid or has expired. Resolution: Verify the token's expiry and audience claim (the `aud` claim must match the broker Dataverse environment URL). Request a new token if expired.

Issue: "Cannot access cs_envelopes" Cause: Missing security role assignment or incorrect environment URL. Resolution: Verify the Application User exists in Dataverse and is assigned the Client Access security role. Test the identity using a WhoAmI call to the Dataverse API.

Issue: "Envelope stuck in Draft, won't send" Cause: Missing signers or documents. Resolution: Retrieve the envelope with `$expand=cs_envelope_signer,cs_envelope_document` and verify at least one signer and one document exist. Confirm the envelope status is "Draft" before retrying `cs_SendEnvelope`.

Issue: "Signer not receiving email" Cause: Email in spam, or envelope has not yet been sent to Nintex. Resolution: Check the envelope status and `cs_nintexenvelopeid`. If status is "InProcess" and a Nintex ID exists, the email was dispatched by Nintex — check the signer's spam folder. Retrieve the signing link from `cs_signinglink` and deliver it via an alternative channel (Teams, SMS).

Issue: High API latency Cause: Large document payloads or Dataverse throttling. Resolution: Use `$select` to limit returned fields; compress documents before Base64 encoding if >5 MB; monitor the `x-ms-ratelimit-burst-remaining-xrm-requests` and `x-ms-ratelimit-time-remaining-xrm-requests` response headers and implement exponential backoff if throttled.

G.2 Common Issues (Portal and Conversion Service)

Issue: Word-to-PDF conversion fails Cause: LibreOffice container crashed or network timeout. Resolution: Restart the Azure Container Instance and inspect container logs. If persistent, check whether the container has sufficient CPU/memory resources.

Issue: Fields not detected after upload Cause: Incorrect placeholder syntax or non-UTF-8 encoding. Resolution: Confirm the placeholder format exactly matches `{{TYPE:Label:Signer}}`. Ensure the document is saved in UTF-8 encoding. Check for hidden characters within placeholders.

Issue: Field positions incorrect on signed document Cause: Coordinate conversion error or non-standard font metrics. Resolution: Use standard fonts (Arial, Times New Roman). Avoid complex nested table layouts. Use the manual position adjustment controls in the field mapping UI to correct positions before submission.

G.3 Support Escalation Matrix

Severity	Response Time	Escalation Path
P1 – Critical	15 minutes	L1 → L2 → Platform Lead → Incident Commander
P2 – High	2 hours	L1 → L2 → Technical Lead
P3 – Medium	8 hours	L1 → L2
P4 – Low	24 hours	L1

Severity Definitions:

- P1: Complete service outage or confirmed security breach
- P2: Significant degradation affecting multiple clients or portal users
- P3: Single client or user impacted; workaround available
- P4: Minor issue or enhancement request

G.4 Vendor Support Contacts

Vendor	Service	Support Number	Account ID
Microsoft	Power Platform	1-800-XXX-XXXX	[FILL]
Microsoft	Azure Support	1-800-XXX-XXXX	[FILL]
Nintex	AssureSign	1-866-XXX-XXXX	[FILL]

Annex H: Glossary

Term	Definition
Application User	Dataverse user identity mapped to a service principal for API access
Broker	Intermediary service between client applications and Nintex AssureSign
CLS	Column-Level Security — restricts access to specific fields in Dataverse
Dataverse	Microsoft's PaaS database platform within the Power Platform ecosystem
DLP	Data Loss Prevention — policies to prevent data exfiltration between connectors
Envelope	A signature request containing one or more documents and signers
LibreOffice Online	Open-source office suite with a headless conversion API; used for Word-to-PDF conversion
MSI	Managed Service Identity — Azure mechanism for service-to-service authentication without secrets
OData	Open Data Protocol — REST-based data access standard used by Dataverse
Placeholder	Template syntax (`{{TYPE:Label:Signer}}`) embedded in Word/PDF documents to mark signature field positions
Power Pages	Microsoft Power Platform product for building low-code external-facing web portals
RLS	Row-Level Security — restricts access to specific records based on the ownerid field
RPO	Recovery Point Objective — maximum acceptable data loss window
RTO	Recovery Time Objective — maximum acceptable downtime window
Service Principal	Non-human Entra ID identity used for programmatic API access
TDE	Transparent Data Encryption — automatic Dataverse database encryption

Annex I: Acronyms

Acronym	Full Form
ACI	Azure Container Instance
API	Application Programming Interface

Acronym	Full Form
CAB	Change Advisory Board
CRUD	Create, Read, Update, Delete
DR	Disaster Recovery
EC	Elections Canada
ERD	Entity Relationship Diagram
ITSG	IT Security Guidance
MFA	Multi-Factor Authentication
MTBF	Mean Time Between Failures
MTTR	Mean Time To Repair
OIDC	OpenID Connect
PaaS	Platform as a Service
PIA	Privacy Impact Assessment
PII	Personally Identifiable Information
RBAC	Role-Based Access Control
REST	Representational State Transfer
RLS	Row-Level Security
SaaS	Software as a Service
SLA	Service Level Agreement
TLS	Transport Layer Security
TRA	Threat and Risk Assessment
UAT	User Acceptance Testing
WAF	Web Application Firewall